KLE Technological University | Creating Value, Leveraging Knowledge

# Design and Implementation of a 4-Bit Unsigned Multiplier Using Verilog HDL on Artix-7 FPGA

NAME : SANDEEP JOSHI USN :- 01FE24BEC039 ROLL NO:- 226 DIV:- B

---

## 1. Introduction

Multiplication is a fundamental arithmetic operation widely used in digital systems such as processors, digital signal processing units, and embedded systems. Implementing multiplication in hardware provides higher speed compared to software-based approaches.

This project presents the design, simulation, and FPGA implementation of a 4bit unsigned multiplier using Verilog Hardware Description Language (HDL). The design is based on the partial product generation and addition method, which follows the conventional binary multiplication process. The designed multiplier is simulated using a Verilog testbench and further synthesized and implemented on a Xilinx Artix-7 FPGA.

---

## 2. Key Features

- 4-bit × 4-bit unsigned multiplication
- 8-bit product output
- Partial product generation using AND logic
- Synthesizable Verilog code
- Verified through simulation
- Implemented and tested on Artix-7 FPGA

---

## 3. Design Specifications

- Input A: 4-bit unsigned binary number
- Input B: 4-bit unsigned binary number
- Output P: 8-bit unsigned binary product
- Design Type: Combinational circuit

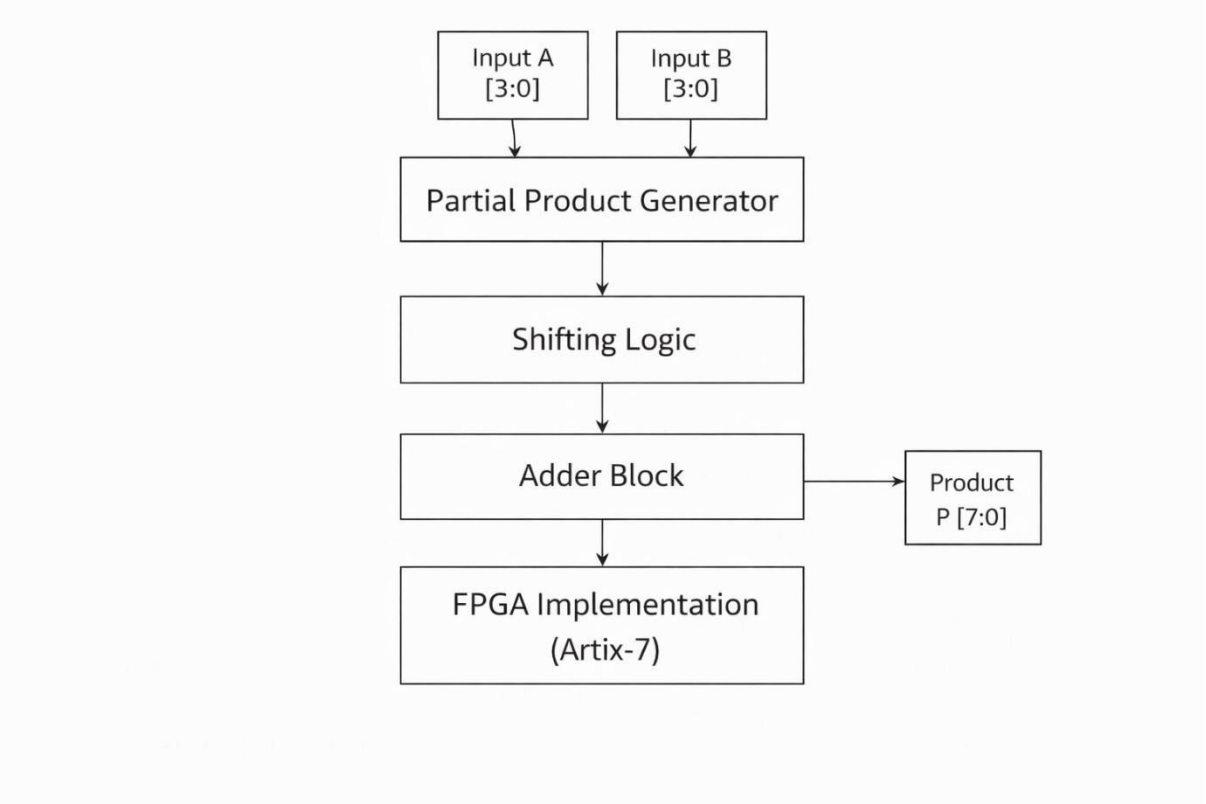
- Multiplication Technique: Partial product addition

---

4. Block Diagram Description

The 4-bit unsigned multiplier consists of the following functional blocks:

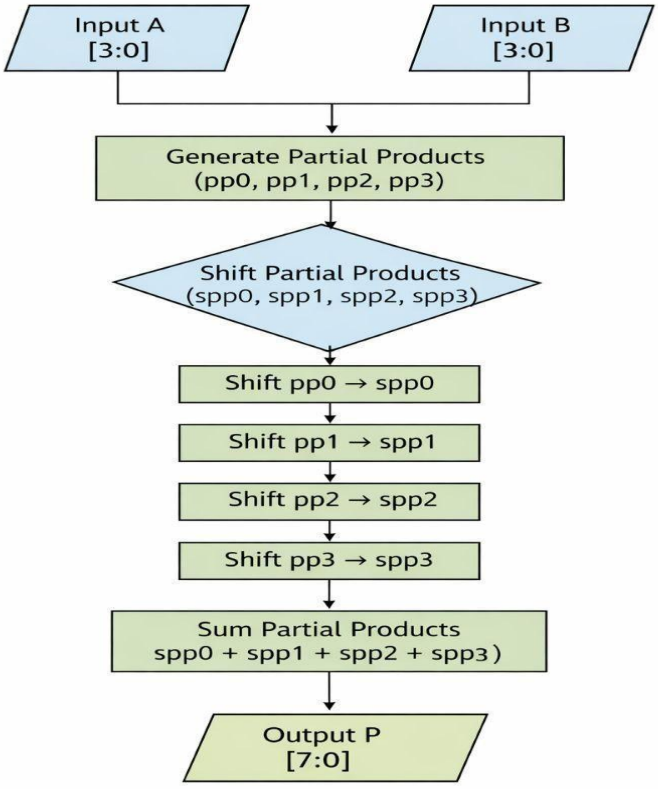
1. Partial Product Generator
   Generates partial products by performing bitwise AND operation between the multiplicand and each bit of the multiplier.
2. Shifting Logic
   Shifts the partial products according to their corresponding bit positions.
3. Adder Block
   Adds all the shifted partial products to obtain the final product.
4. FPGA Implementation Block
   The complete design is synthesized and mapped onto the Artix-7 FPGA using configurable logic blocks.




---

5. Algorithm

1. Accept 4-bit unsigned inputs A and B.
2. Generate partial products using AND operations.
3. Shift each partial product according to the bit position of B.
4. Add all shifted partial products.
5. Produce the final 8-bit multiplication result P.
6. Implement and verify the design on Artix-7 FPGA.




---

6. Verilog Code Analysis

The Verilog design uses continuous assignments to implement a combinational multiplier.

- Partial products are generated by ANDing input A with each bit of input B.
- Shifting is achieved through bit concatenation.
- All shifted partial products are added together to generate the final output.

This approach accurately models binary multiplication in hardware and is fully synthesizable for FPGA implementation.

```
module multi(input  [3:0] A, input  [3:0] B, output [7:0] P);

    wire [3:0] pp0, pp1, pp2, pp3;
    wire [7:0] spp0, spp1, spp2, spp3;

    assign pp0 = A & {4{B[0]}};
    assign pp1 = A & {4{B[1]}};
    assign pp2 = A & {4{B[2]}};
    assign pp3 = A & {4{B[3]}};

    assign spp0 = {4'b0000, pp0};
    assign spp1 = {3'b000,  pp1, 1'b0};
    assign spp2 = {2'b00,   pp2, 2'b00};
    assign spp3 = {1'b0,    pp3, 3'b000};


    assign P = spp0 + spp1 + spp2 + spp3;

endmodule
```

---

7. Testbench Description

A Verilog testbench is developed to verify the functionality of the multiplier before FPGA implementation.

- Instantiates the multiplier module as the Device Under Test (DUT).
- Applies different combinations of input values.
- Uses the $monitor statement to observe simulation results.
- Verifies correctness for normal, maximum, and zero input cases.

```
module multb();

    reg [3:0] A;
    reg  [3:0] B;
    wire [7:0] P;

    multi uut (.A(A),.B(B),.P(P));

    initial begin

        A = 4'd3; B = 4'd5;
        #10;

        A = 4'd7; B = 4'd4;
        #10;

        A = 4'd9; B = 4'd9;
        #10;

        A = 4'd15; B = 4'd15;
        #10;

        A = 4'd0; B = 4'd10;
        #10;

    end

endmodule
```

---

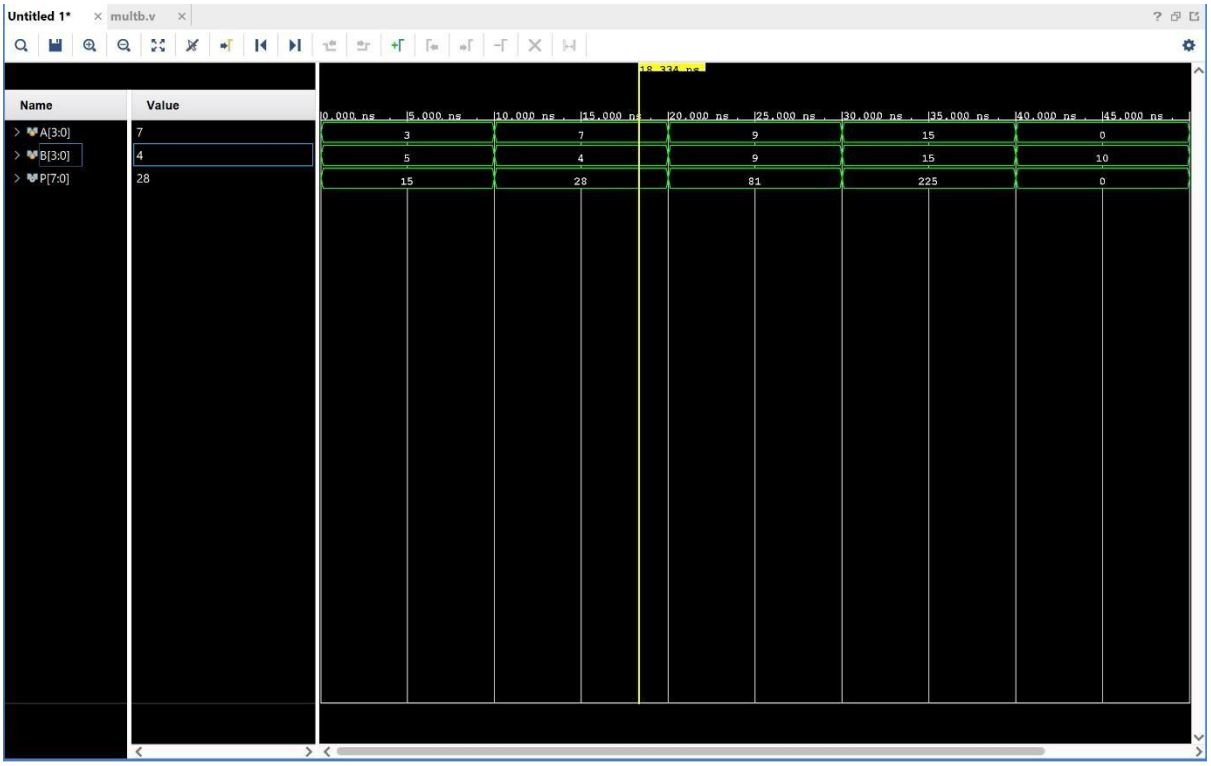
8. Simulation and FPGA Implementation Results

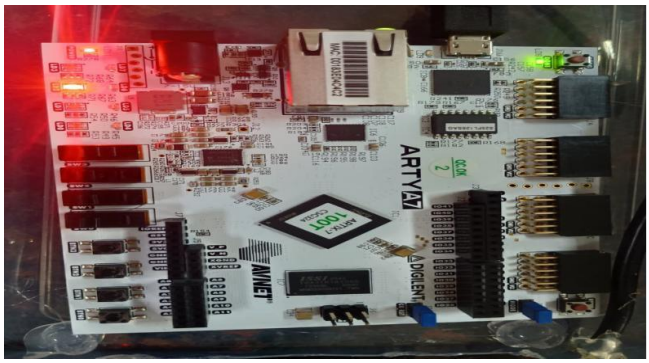
Simulation Results

- All applied test cases produced correct output values.
- Functional verification was successful with no errors.

FPGA Implementation Results

- The design was synthesized and implemented successfully on the Xilinx Artix-7 FPGA using Vivado.
- The output observed on the FPGA matched the expected multiplication results.
- The design utilized minimal FPGA resources due to its simple combinational structure.



---

## 9. Applications

- Arithmetic Logic Units (ALUs)

- Microprocessors and controllers
- Digital signal processing systems
- FPGA-based arithmetic circuits
- Educational digital design projects

---

10. Advantages

- Simple and easy to understand design
- High-speed operation due to combinational logic
- No clock dependency
- Successfully implemented on Artix-7 FPGA
- Scalable to higher bit-width multipliers

---

11. Limitations

- Hardware complexity increases for higher bit widths
- No power optimization techniques used
- Not suitable for large multipliers without architectural optimization

---

12. Synthesis Report (simple)

```
#-----------------------------------------------------------
# Vivado v2025.1 (64-bit)
# SW Build 6140274 on Thu May 22 00:12:29 MDT 2025
# IP Build 6138677 on Thu May 22 03:10:11 MDT 2025
# SharedData Build 6139179 on Tue May 20 17:58:58 MDT 2025
# Start of session at: Thu Jan  1 21:12:09 2026
# Process ID         : 9940
# Current directory  : C:/Users/ambig/Vivado/unsigned.runs/synth_1
# Command line       : vivado.exe -log multi.vds -product Vivado -mode batch -messageDb vivado.pb -notrace -
# Log file           : C:/Users/ambig/Vivado/unsigned.runs/synth_1/multi.vds
# Journal file       : C:/Users/ambig/Vivado/unsigned.runs/synth_1\vivado.jou
# Running On         : LAPTOP-3SUQPA02
# Platform           : Windows Server 2016 or Windows 10
# Operating System   : 26200
# Processor Detail   : 13th Gen Intel(R) Core(TM) i5-13500H
# CPU Frequency      : 3187 MHz
# CPU Physical cores : 12
# CPU Logical cores  : 16
# Host memory        : 16852 MB
# Swap memory        : 1879 MB
# Total Virtual      : 18731 MB
# Available Virtual  : 2939 MB
#-----------------------------------------------------------
(source and full synthesis log text exactly as provided)
```

---

Reports | Design Runs | **Power** ×

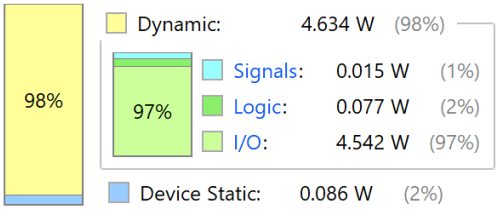
**Summary**

Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

| | |
|---|---|
| **Total On-Chip Power:** | **4.72 W** |
| **Design Power Budget:** | **Not Specified** |
| **Process:** | typical |
| **Power Budget Margin:** | **N/A** |
| **Junction Temperature:** | **47.6°C** |
| Thermal Margin: | 37.4°C (7.8 W) |
| Ambient Temperature: | 25.0 °C |
| Effective ϑJA: | 4.8°C/W |
| Power supplied to off-chip devices: | 0 W |
| Confidence level: | Low |

Launch Power Constraint Advisor to find and fix invalid switching activity

**On-Chip Power**




**Design Timing Summary**

| Setup | | Hold | | Pulse Width | |
|---|---|---|---|---|---|
| Worst Negative Slack (WNS): | inf | Worst Hold Slack (WHS): | inf | Worst Pulse Width Slack (WPWS): | NA |
| Total Negative Slack (TNS): | 0.000 ns | Total Hold Slack (THS): | 0.000 ns | Total Pulse Width Negative Slack (TPWS): | NA |
| Number of Failing Endpoints: | 0 | Number of Failing Endpoints: | 0 | Number of Failing Endpoints: | NA |
| Total Number of Endpoints: | 20 | Total Number of Endpoints: | 20 | Total Number of Endpoints: | NA |

**There are no user specified timing constraints.**

## 12. Conclusion

In this project, a 4-bit unsigned multiplier was designed using Verilog HDL based on the partial product addition method. The design was successfully simulated and verified using a testbench. Furthermore, the multiplier was synthesized and implemented on a Xilinx Artix-7 FPGA, demonstrating correct hardware functionality. This project provides a strong foundation for understanding hardware-based multiplication and FPGA implementation.